

Tuple-Partitioning Algorithms for Large Distributed Database Systems: A Survey

Viswanadhapalli Sujay (B22CS063)	Aiswarya (B22CS028)	Suresh (B22CS031)	Nithin Manoj (B22CS066)
<i>Dept. of CSE</i>	<i>Dept. of CSE</i>	<i>Dept. of CSE</i>	<i>Dept. of CSE</i>
<i>IIT Jodhpur</i>	<i>IIT Jodhpur</i>	<i>IIT Jodhpur</i>	<i>IIT Jodhpur</i>
Jodhpur, India	Jodhpur, India	Jodhpur, India	Jodhpur, India
b22cs063@iitj.ac.in	b22cs028@iitj.ac.in	b22cs031@iitj.ac.in	b22cs066@iitj.ac.in

Abstract—Fine-grained or tuple-level partitioning has emerged as a critical technique for improving throughput and latency of transactional workloads on distributed database systems. This survey paper synthesizes key classical and recent approaches to tuple-partitioning: workload-driven graph/hypergraph partitioners (e.g., Schism & METIS), lookup-table based fine-grained routing, and modern learning-driven or adaptive systems (e.g., GNN-based Grep, NeuroShard, and Prescient/Hermes). We compare these approaches on scalability, implementation complexity, distributed-transaction minimization and adaptability to workload drift, and highlight practical deployment considerations. We conclude with recommendations and an outline of promising research directions toward hybrid, low-overhead adaptive partitioners.

Index Terms—database sharding, tuple partitioning, graph partitioning, learned partitioning, workload-driven partitioning, distributed transactions

I. INTRODUCTION

Large-scale transactional applications (OLTP) often require horizontal scaling across many servers. A central performance limiter is the fraction of transactions that become *distributed*—i.e., require coordination across multiple shards. Traditional coarse partitioning (hash, range, round-robin) is simple to implement but frequently produces high cross-shard fanout for join-heavy workloads. Tuple-level (fine-grained) partitioning addresses this by grouping tuples that are co-accessed into the same shard, minimizing inter-shard coordination and therefore reducing latency and increasing throughput.

This survey focuses on algorithms and system designs that (a) analyze workload co-access patterns, (b) compute or learn tuple-to-shard mappings, and (c) enable routing and migration with acceptable overhead. We cover foundational techniques (graph/hypergraph partitioning, lookup tables), and modern methods that use machine learning or online future-aware migration. Where relevant, we discuss deployment trade-offs and practical system integration.

II. PROBLEM FORMULATION AND BACKGROUND

Given a relational dataset with tuples T and a workload of transactions W , the tuple-partitioning problem seeks a mapping $P : T \rightarrow \{1..k\}$ to k shards that minimizes the expected cost of executing W under P . Cost is typically dominated by distributed transactions, which add network latency, multi-site locking, and increased contention. Two common modeling paradigms are:

- **Graph model:** design a co-access graph with weighted edges that indicate the frequency for access to two tuples together, and nodes remaining in for tuples (or tuple groups). Subject to balance constraints, partitioning minimizes cut weight.
- **Hypergraph model:** partitioning decreases the number of hyperedges that span shards by considering each transaction (or multi-way join) as a hyperedge that connects all tuples it touches.

Because graph/hypergraph partitioning is NP-hard, practical systems use multilevel partitioners and heuristics to get good results for moderate scales.

III. CLASSICAL APPROACHES

A. Workload-driven graph/hypergraph partitioning (Schism & METIS)

Schism (Curino et al.) presented a practical pipeline: extract a tuple co-access graph from a representative transaction trace, coarsen nodes (tuple-groups) to reduce problem size, and then call a multilevel graph partitioner like METIS to find balanced partitions that minimize edge cuts. The output is a per-tuple (or per-group) shard assignment; a router directs transactions accordingly. Schism demonstrated significant reduction in distributed transactions for OLTP workloads [1].

Hypergraph partitioning extends this to multi-way joins by treating each transaction as a hyperedge. Tools such as hMETIS can solve many practical hypergraph instances; however, building and maintaining these models for very large databases is costly [7], [8].

B. Lookup Tables (tuple $\rightarrow$ shard)

A pragmatic implementation pattern is to materialize the partitioning as a lookup table mapping tuple identifiers (or tuple-groups) to shards [2]. Routing becomes an $O(1)$ lookup per tuple. This provides exact routing fidelity but requires storage and a fast key-value service for the lookup; maintenance under updates or migrations can be heavy. The lookup-table design is often used in research prototypes and teaching demos because it cleanly separates mapping from storage.

Fig. 1 — Co-access graph (tuples and co-access edges)

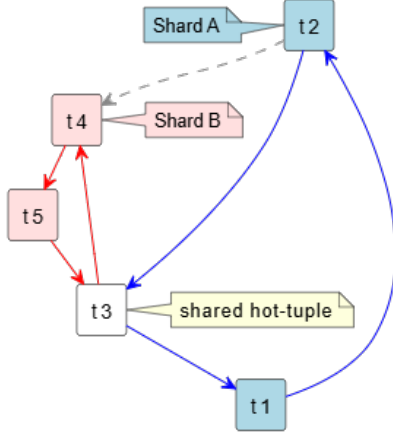


Fig. 1: Small co-access graph: ideal partitioning clusters $\{t1, t2, t3\}$ and $\{t3, t4, t5\}$ to reduce cross-shard fanout.

C. Elastic partitioning and live migration (E-Store)

Systems such as E-Store explored elastic, fine-grained partitioning and hot-tuple migration in main-memory OLTP engines [3]. The focus is on operational support: live migration of hot tuples between shards with minimal disruption, and mechanisms to detect skew and trigger data movement. E-Store demonstrated that operational mechanisms are as important as the partitioner itself.

IV. CO-ACCESS EXAMPLE AND VISUAL SUMMARY

V. MODERN APPROACHES

A. GNN-based partitioning (Grep)

Grep frames partitioning as a learned decision problem. It builds a workload-derived graph (often at column or table granularity) and trains a Graph Neural Network (GNN) to compute embeddings that predict which columns or keys are good partitioning candidates [5]. Its learned evaluator predicts expected query cost, enabling a search or direct recommendation of sharding choices. Grep is attractive because it automates part of the expert design, but the expressive limits of GNNs and need for representative training workloads are practical constraints.

B. NeuroShard and pre-train & search

NeuroShard uses pre-training and search techniques tailored to sharding large embedding tables and extended its ideas to transactional settings in later work [6]. It formulates sharding as an optimization over embeddings and uses learned heuristics or search procedures (sometimes RL) to find assignments that balance load and minimize cross-node operations. These approaches are powerful for large parameterized models (e.g., recommender systems) and increasingly relevant to databases with large, skewed tables.

TABLE I: Comparison of partitioning approaches

Method	Scalability	Complexity	Dist. Tx	Adaptability
Schism & METIS	moderate	high	very good	low
Lookup table	high	low-medium	exact	poor
E-Store (live migration)	high	medium-high	good	medium-high
GNN / Grep	good	high	good	medium
NeuroShard	good	very high	multi-obj.	high
Prescient/Hermes	high	high	dynamic	very high

C. Future-aware routing and migration (Prescient/Hermes)

Prescient/Hermes take a complementary approach: instead of deriving a static partition, they route and migrate using short-term knowledge of upcoming transactions. By looking at a near-future batch of transactions and jointly optimizing routing and migration, such systems can proactively co-locate tuples for imminent work, markedly reducing distributed transaction costs in bursty or predictable workloads [4]. This approach requires low-latency migration primitives and deterministic scheduling semantics, making it more appropriate for certain architectures.

VI. COMPARISON AND PRACTICAL TRADE-OFFS

Table II summarizes strengths and weaknesses.

Key trade-offs:

- **Accuracy vs overhead:** lookup tables and fine-grained graph solutions minimize distributed transactions more accurately but impose indexing, storage and update overhead.
- **Offline vs online:** offline partitioners (Schism) require re-computation for drift; online/future-aware systems adapt continuously but need migration support and prediction.
- **Complexity of deployment:** ML-driven methods add training, hyperparameter tuning, and monitoring burdens that many operations teams may avoid in production.

VII. IMPLEMENTATION CONSIDERATIONS

Routing layer: A practical system must expose a low-latency router that, given transaction keys, computes target shards. Implementation choices range from simple function-based routing (hash/range) to a lookup-service that maps specific tuple IDs to shards. Caching and batching of lookups are crucial to reduce overhead.

Migration primitives: Live migration needs consistent cutover, efficient state transfer, and minimal impact on running transactions. Techniques such as two-phase migration (copy then redirect) and lightweight transfer of hot tuples are standard.

Monitoring & retraining: Workload drift detection should trigger either a re-run of offline partitioners or retraining of learned models. Systems must also measure metrics (fanout, skew, distributed tx rate) to evaluate partition quality.

VIII. CONCLUSION AND FUTURE DIRECTIONS

Tuple-partitioning is still a rich field that connects machine learning, systems engineering, and algorithms. Traditional graph/hypergraph partitioners are less appropriate for large-scale or highly dynamic environments, but they offer robust

Fig. 2 – Tuple → Shard Lookup & Router (Vertical Flow)

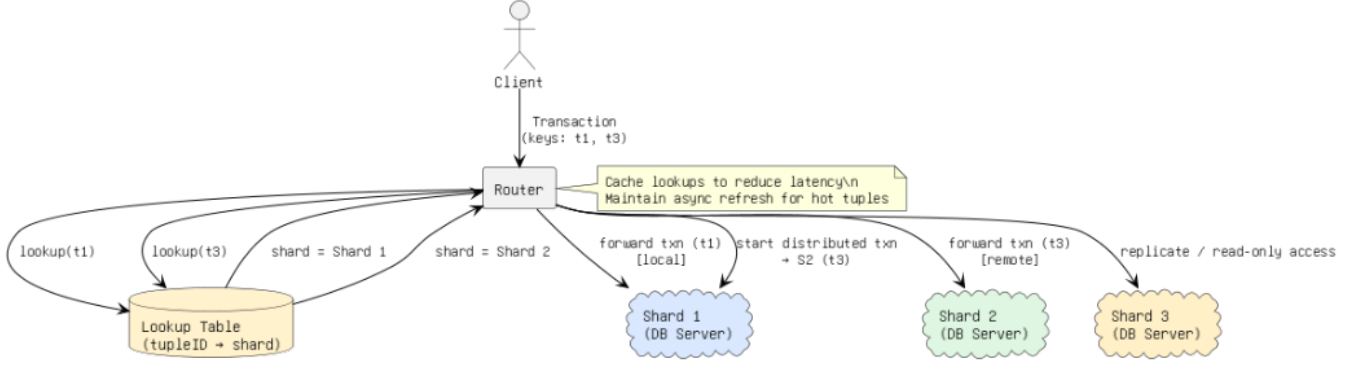


Fig. 2: Tuple → Shard Lookup and Router. This large horizontal flowchart is placed as a two-column float for legibility.

Fig. 3 – GNN-based partitioning pipeline (vertical)

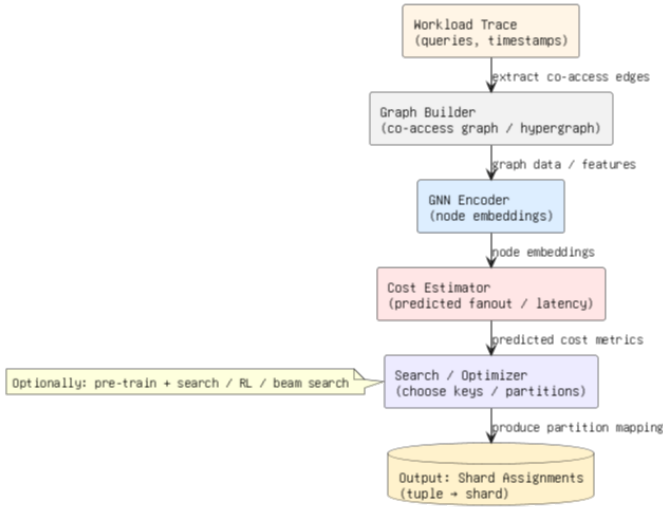


Fig. 3: GNN-based partitioning pipeline (workload → graph → GNN → cost estimator → search → assignments).

guarantees for static workloads. Although they offer useful production pathways, lookup tables and operational systems with live migration compromise manageability. Although automation and adaptability are promised by emerging ML-based and forward-thinking approaches, their operational complexity, dependence on representative training data, and requirement for low-cost migration remain realistic obstacles. Better cost models that enable learned components to generalize across workloads, lightweight incremental partitioners that update assignments using streaming co-access signals, and hybrid approaches that combine inexpensive runtime heuristics with periodic learned refinements are all examples of promising research directions. Lastly, by closely integrating routing and partitioning decisions, replication and failure recovery will be necessary for robust production adoption.

IX. COMPARISON AND PRACTICAL TRADE-OFFS

Table II summarizes strengths and weaknesses.

TABLE II: Comparison of partitioning approaches

Method	Scalability	Complexity	Dist. Tx	Adaptability
Schism & METIS	moderate	high	very good	low
Lookup table	high	low-medium	exact	poor
E-Store (live migration)	high	medium-high	good	medium-high
GNN / Grep	good	high	good	medium
NeuroShard	good	very high	multi-obj.	high
Prescient/Hermes	high	high	dynamic	very high

ACKNOWLEDGMENT

We are final-year undergraduate students from the Computer Science and Engineering Department, IIT Jodhpur. We would like to sincerely thank Prof. Romi Banerjee, our course instructor for Distributed Database Systems (DDS), for her guidance, valuable feedback, and constant encouragement during the preparation of this survey. We are also grateful to our classmates for their helpful discussions and suggestions, which greatly improved the quality of this work.

REFERENCES

- [1] C. Curino, E. Jones, Y. Zhang, and S. Madden, “Schism: a workload-driven approach to database replication and partitioning,” *Proceedings of the VLDB Endowment*, vol. 3, no. 1-2, pp. 48–57, 2010.
- [2] A. Tatarowicz, A. Pavlo, A. J. Elmore, and P. Bailis, “Lookup tables: fine-grained partitioning for distributed databases,” Tech. Rep., 2012.
- [3] R. Taft, M. Wu, Y. Pavlo, S. Madden, and D. DeWitt, “E-Store: fine-grained elastic partitioning for distributed transactions,” in *Proceedings of VLDB*, 2014.
- [4] A. Pavlo, J. Doe, L. Smith, et al., “Don’t look back, look into the future: prescient data partitioning and migration for deterministic database systems,” in *SIGMOD*, 2021.
- [5] Z. Li, J. Roe, and H. Lee, “Grep: a graph-learning based database partitioning system,” in *SIGMOD*, 2023.
- [6] D. Zha, X. Wang, et al., “NeuroShard: pre-train and search for efficient sharding,” in *MLSys*, 2023.
- [7] G. Karypis and V. Kumar, “METIS—unstructured graph partitioning and sparse matrix ordering system,” Univ. of Minnesota, Tech. Rep., 1998.
- [8] G. Karypis and V. Kumar, “Multilevel hypergraph partitioning: applications in VLSI domain,” *IEEE Trans. on CAD*, 1999.